Thread 3:

```
4, 6
```

Each thread calculates a partial result.
Then all partial results are combined to get the final answer.

Why Parallel Reduction is Used?
Parallel reduction is used because:
- It reduces execution time
- It uses multiple threads
- It is useful for large arrays
- It improves performance

Min Reduction
Minimum reduction finds the smallest element.
Example:

```
Array = [10, 4, 8, 2, 15]
```

Output:

```
Minimum = 2
```

In parallel:
- Each thread finds local minimum
- Final minimum is calculated from all local minimum values

Max Reduction
Maximum reduction finds the largest element.
Example:

```
Array = [10, 4, 8, 2, 15]
```

Output:

```
Maximum = 15
```

In parallel:
- Each thread finds local maximum
- Final maximum is calculated from all local maximum values

Sum Reduction
Sum reduction adds all elements.
Example:

```
Array = [10, 4, 8, 2, 15]
```

Output:

```
Sum = 39
```

In parallel:
- Each thread calculates partial sum
- All partial sums are added to get final sum

Average Reduction
Average is calculated using sum.
Formula:

```
Average = Sum / Number of elements
```

Example:

```
Array = [10, 4, 8, 2, 15]

Sum = 39
Number of elements = 5

Average = 39 / 5 = 7.8
```

In parallel:
- First calculate sum using parallel reduction
- Then divide sum by total number of elements

OpenMP Reduction Clause
OpenMP provides a special reduction clause.
Syntax:

```
#pragma omp parallel for reduction(+:sum)
```

This means:
- Each thread gets its own private copy of sum
- Each thread calculates partial sum
- At the end, OpenMP combines all sums automatically

For min:

```
#pragma omp parallel for reduction(min:minVal)
```

For max:

```
#pragma omp parallel for reduction(max:maxVal)
```

b) CUDA Based Vector Operations
1. What is CUDA?
CUDA (Compute Unified Device Architecture) is a parallel computing platform developed by NVIDIA.
It allows programs to run on **GPU instead of CPU**.

2. Why CUDA is used?
- GPU has many cores (thousands)
- Can process large data **in parallel**
- Faster than CPU for vector operations

3. Problem Statement
Given a large vector of size **N**, find:
- Maximum element
- Minimum element
- Arithmetic mean
- Standard deviation

4. Key Concepts in CUDA
🔹 **Kernel**
- A function that runs on GPU
- Called from CPU

```
__global__ void functionName()
```

🔹 **Threads**
- Small workers inside GPU
- Each thread processes part of data

🔹 **Blocks**
- Group of threads

🔹 **Grid**
- Collection of blocks

🔹 **Execution Model**

```
CPU → launches kernel → GPU executes parallel threads
```

5. Steps to Solve Problem
Step 1: Input
- Take size **N**
- Generate or input vector

Step 2: Transfer Data
- Copy data from CPU to GPU memory

Step 3: Kernel Execution
Each thread processes one element:
- Compare for max/min
- Add for sum

Step 4: Reduction
Combine results:
- Final max
- Final min
- Final sum

Step 5: Mean Calculation
Formula:

```
Mean = Sum / N
```

Step 6: Standard Deviation
Steps:
1. Calculate mean
2. For each element:

```
(x - mean)^2
```

3. Sum all values
4. Divide by N
5. Take square root

Formula:

```
Std Dev = √( Σ(x - mean)² / N )
```

6. Performance Measurement
- Measure time using GPU timers or clock
- Compare CPU vs GPU

7. Advantages of CUDA
- High performance
- Faster execution for large datasets
- Efficient parallel computation

8. Limitations
- Requires NVIDIA GPU
- Complex programming compared to OpenMP

Q) Assignment : Vector and Matrix Operations Design parallel algorithm to Add two large vectors. Multiply Vector and Matrix. Multiply two N x N arrays using n2 processors.

Vector and Matrix Operations (Parallel Algorithm)

1. Vector Addition
Concept:

Two vectors are added element by element.

C[i] = A[i] + B[i]
Parallel Idea:
Each element is independent
So multiple threads can compute different elements at the same time
Example:
A = [1,2,3]
B = [4,5,6]

C = [5,7,9]

2. Vector-Matrix Multiplication
Concept:

A vector is multiplied with a matrix.

C[i] = Σ (A[j] * B[j][i])
Parallel Idea:
Each output element can be computed independently
Assign one thread per element
Example:
Vector: [1,2]
Matrix:
[3,4]
[5,6]

Result:
[13,16]

3. Matrix Multiplication (N × N)
Concept:

Multiply two matrices.

C[i][j] = Σ (A[i][k] * B[k][j])

Parallel Idea:
Each element of result matrix is independent
Use one thread per element

n² Processors Concept
For N × N matrix → N² elements
Use N² processors → one processor per element

Each processor computes one C[i][j]

Advantages of Parallel Approach
- Faster computation
- Efficient for large data
- Uses multiple cores